Knowledge and Information Systems (2025) 67:9615–9636
https://doi.org/10.1007/s10115-025-02516-0

RESEARCH

Comparative analysis of sentence similarity detection using machine and deep learning with vectorization techniques

Gayatri Girish Asalkar[1] · Bechoo Lal[1] · Nilesh B. Korade[2]

Received: 28 January 2025 / Revised: 24 April 2025 / Accepted: 5 June 2025 /
Published online: 9 July 2025
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2025

Abstract
Sentence similarity detection provides significant advantages across different applications, such as customer support applications, e-commerce customer service, educational platforms, community opportunities and question-answering systems. This study presents a comparative analysis of various machine learning and deep learning models for sentence similarity detection, including cosine similarity, adaptive boosting (AdaBoost), Extreme Gradient Boosting (XGBoost), Convolutional Neural Network with Long Short-Term Memory, and Bidirectional Encoder Representations from Transformers with Long Short-Term Memory. This research also evaluates the impact of various vectorization techniques, such as Term Frequency-Inverse Document Frequency, OpenAI embeddings and Topic Modeling, on the performance of these models. The proposed research validates the effectiveness of these approaches in enhancing the accuracy of similarity detection. The findings offer valuable insights into an optimal combination of vectorization methods and models for improved sentence similarity detection.

Keywords Sentence similarity · Vectorization · Quora dataset · Topic model · OpenAI · Embedding · Adaboost · And XGboost

1 Introduction

In natural language processing (NLP), phrase resemblance recognition is a critical component for assessing the degree of similarity among two texts [1]. This method is beneficial for a

✉ Gayatri Girish Asalkar
gayatri.teke@gmail.com

Bechoo Lal
blal2k7@gmail.com

Nilesh B. Korade
nilesh.korade.ml@gmail.com

1 Department of Computer Science and Engineering, Shri Jagdishprasad Jhabarmal Tibrewala University, Jhunjhunu, Rajasthan 333001, India

2 Department of Computer Science and Engineering (Artificial Intelligence), Vishwakarma Institute of Technology, Pune 411037, India

wide range of applications, as well as machine conversion, paraphrase detection, information retrieval, question answering, and content summarization [2]. The traditional sentence similarity detection systems like bag-of-words (BoW) and cosine similarity are mainly focused on simple semantic meanings. These models often struggle to capture the deep meaning among the sentences and lack the difficulty to recognize context or delicate shades of meaning [3].

Transformer-based architectures, particularly Bidirectional Encoder Representations from Transformers (BERT) and its alternates, have shown exceptional proficiency in grasping the complex semantics of natural language [4]. These models operate by acquiring contextualized word representations, which significantly improve their capacity to discern word meanings in relation to their surrounding context. This ability to model context has revolutionized sentence similarity detection, allowing for more accurate comparisons between sentences, even when their lexical structures vary [5, 6].

A notable development in this field has been the emergence of pre-trained sentence embeddings (SE), such as those created by OpenAI [7]. The generative pre-trained transformer (GPT) [8] models, especially GPT-3 [9] and its subsequent versions, have achieved considerable acclaim for a series of NLP tasks, owing to their vast pre-trained knowledge and capability to produce high-grade SE [10]. These embeddings serve as dense vector representations that retain the semantic essence of sentences, enabling similarity measurement using techniques, such as cosine similarity or other distance metrics [11, 12].

The combination of transformer-based deep learning models with OpenAI's embeddings is transforming various industries, including legal text analysis, healthcare document assessment, chatbot development, and content recommendation systems [13, 14]. As transformer models progress and pre-trained embeddings become more widely available, the correctness and efficacy of sentence similarity recognition are expected to enhance significantly, leading to groundbreaking advancements in AI-driven applications [15–19].

1.1 Motivation

The increasing demand for accurate sentence similarity detection across various applications, such as customer service, e-commerce and educational platforms, has driven this research. A key challenge in these domains is the frequent repetition of user queries, which reduces response quality and negatively impacts user experience. This issue is particularly prevalent in customer service chatbots, where redundant inquiries lead to inefficiencies and delays in retrieving relevant responses. To address this challenge, this study develops an automated sentence similarity detection system that enhances user experience by effectively matching user queries with relevant responses. The research explores multiple vectorization techniques, including Term Frequency-Inverse Document Frequency (TF-IDF), Correlated Topics Models (CTM), and OpenAI embeddings, to convert textual data into meaningful vector representation. Moreover, it implements and evaluates several machine learning and deep learning models, such as Cosine Similarity (CS), Adaptive Boosting (AdaBoost), Extreme Gradient Boosting (XGBoost), Convolutional Neural Network-Long Short-Term Memory (CNN-LSTM), and Bidirectional Encoder Representations from Transformers-Long Short-Term Memory (BERT-LSTM) to attain high-accuracy sentence similarity detection. The main contribution of this research is described below,

- Development of an automated sentence similarity detection system for enhancing information retrieval in applications, such as question-answering systems, customer support chatbots, and community forums.

- Comparative analysis of vectorization techniques, including TF-IDF, correlated topic models (CTM) and OpenAI embeddings, to assess their effectiveness in encoding sentence semantics.
- Implementation and performance evaluation of multiple machine learning and deep learning models to define their suitability for sentence similarity detection.

The remainder of the paper is structured as follows: Sect. 2 analyzes analogous work from previous years. Section 3 describes the preferred methodology. Section 4 includes the experimental analysis and discussion of the provided model, while Sect. 5 presents the conclusion and future steps.

2 Literature review

Korade et al. [20] introduced cascaded CNN based on Word2Vec vectorization for detecting questions with the same meaning. Also, this research employed PCA for dimensionality reduction without compromising prediction accuracy, which leads to less training time. The classification algorithms produced effective results compared to other models on the Word2Vec vectorized dataset. Associated with other conventional methods, Cascaded CNN produced a high accuracy of 83%. However, the cascaded CNN takes more time for training, which leads to producing less accurate results.

Automatic similarity detection is important in answering questions. Quora dataset is one of the recent famous datasets used by many researchers in the task of duplicate question-finding systems. Furthermore, the ground truth labels provided in the dataset are not completely accurate and may contain inaccurate labels. Le et al. [21] suggested utilizing Bert, rules, and human-assigned labels to increase the quality of Quora data. When associating the system's labels with those allocated by Mturkers, the suggested strategy attained an accuracy of more than 80%. The current constraint of the study is the possibility of wrongly categorized data in the Quora dataset. This could have a significant influence on the accuracy and dependability of models built with this dataset.

Imtiaz et al. [22] used the Quora Question Pairs dataset for duplicate question pair prediction. Initially, three types of word embeddings were implemented separately to vectorize all questions: Google Word2Vec embeddings, 300-dimensional FastText crawl embeddings, and 300-dimensional FastText crawl subword embeddings. Following that, the dataset's replica questions are anticipated using the Siamese MaLSTM Neural Network (NN) model. Finally, the model is tested with 100,000 question pairs. The results show that the suggested technique attained an accuracy of 91.14%, which is higher than the prevailing models. The limitations of this study are network complexity and dataset size, which are used in performance evaluation.

NIJAGUNARYA [23] developed a new duplicate question detection system by employing some machine learning (ML) classifiers, such as logistic regression, decision tree (DT), adaboost and bagging random forest. Evaluation results show these classifiers produce effective results associated with conventional methods. However, the study did not use any innovative models, which is a restriction.

Wang et al. [24] introduced duplicate questions utilizing DL models, such as convolutional neural networks (CNN), recurrent neural networks (NN), and long short-term memory (LSTM), to find similarity among words. Also, the Word2Vec model is used to convert text into vector format. Based on the Word2Vec model, three DL approaches are used in this research: WV-CNN, RNN, and LSTM. These methods have attained high results in terms of recall rates @5, @10 and @20.

Wang et al. [25] presented a cross-lingual sentence resemblance computation technique based on semantic and organizational aspects of input sentences. Compared to other techniques presented, the pre-trained model employed in this study attained effective results in the authentication and final test set of the data. The smaller size of the dataset is a limitation that leads to overfitting and less performance.

3 Proposed methodology

This study implemented automatic sentence similarity identification to improve the performance of a question-and-answering system. Figure 1 depicts the overall workflow for the suggested approach.

The incoming text is pre-processed with techniques, such as stop word removal, tokenization, and lemmatization. This study introduces Term Frequency-Inverse Document Frequency (TF-IDF), Correlated Topic Model, and Open AI embedding as methods for converting pre-processed text input to numerical vectors. To identify sentence similarity, embeddings generated by several methods are combined with models, such as cosine similarity, Adaboost, XGboost, BERT-LSTM, and CNN-LSTM.

3.1 Pre-processing

The database is assessed for replica or missing entities. Special characters like the percentage symbol (%) were replaced with their matching string representations. Moreover, URLs and HTML elements are eliminated from the dataset. Lastly, chat words like "N.A", which portrays "not applicable", were substituted with appropriate replacements. After tokenization and stop-word removal, the dataset underwent lemmatization [26].

3.2 Word embedding

Word embedding, often called vectorization, is a technique for detecting semantic and syntactic similarities among words. In this phase, words or sentences are signified as real number pathways in a high-dimensional space. For vectorizing the pre-processed words, the suggested solution used TF-IDF, a correlate topic model, and Open AI embedding techniques.

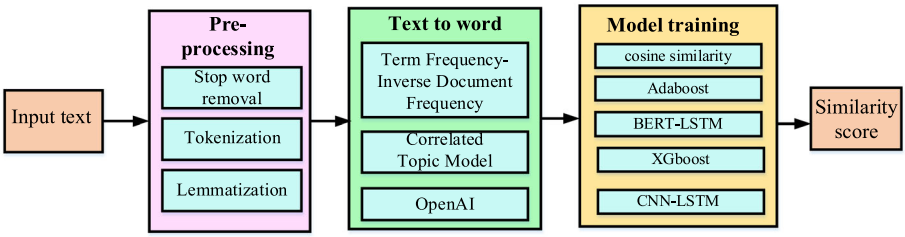

Fig. 1 Overall workflow of the proposed method

3.2.1 Term frequency inverse document frequency

This technique provides data about less and more significant words in the document. The most significant word indicates the additional times a word performs in the manuscript. The term frequency (TF) refers to the ratio of a word's presence in a text to the entire number of words in the manuscript. The TF is mathematically definite as,

$$\mathrm{TF-IDF}(w, d) = \mathrm{TF}(w, d) \times \mathrm{IDF}(w) \tag{1}$$

TF can be denoted as,

$$\mathrm{TF}(w, d) = \frac{\mathrm{TF}(w, d)}{\sum_{w'} f(w', d)} \tag{2}$$

where $f(w, d)$ defines the number of times word w appears in document d and the denominator represents the total word count in d. The term IDF is mathematically denoted as,

$$\mathrm{IDF}(w) = \log\left(\frac{N}{1 + \mathrm{DF}(w)}\right) \tag{3}$$

where N defines the total number of documents and $\mathrm{DF}(w)$ is the number of documents containing the word w. In TF-IDF [27], more frequent words in a document are given more importance, and rare words in documents are given more weight.

3.2.2 Correlated topic model (CTM)

A topic model is defined as a document with a variety of latent themes. LDA is a popular topic model that applies a conjugate Dirichlet prior to the completed issue mixing quantities to make interpretation easier. LDA is commonly utilized in industrial applications since it is simple and scalable. The Dirichlet prior does not account for topical dependencies. The classic CTM elegantly extends LDA by replacing the Dirichlet prior with a logistic-normal prior that captures pairwise topic correlations via the Gaussian covariance matrix. Nevertheless, the enhanced extraction is computationally expensive. The number of constraints in the covariance matrix grows with the square of the number of themes, and the constraint estimate for the entire rank matrix is incorrect in high-dimensional space. More prominently, frequent matrix inversion procedures during inference lead to $O\left(K^3\right)$ time intricacy, which has expressively constrained the data scales. A scalable Gibbs sampling process based on data increase was derived to address this issue. By conveying down the implication cost to $O\left(K^2\right)$ per the document, the calculation is static high for applied real-world tasks. Methodology for Sentence Similarity Prediction using CTM.

To predict sentence similarity using CTM, the following steps are involved:

1. *Topic Distribution Inference* For each sentence, infer its topic distribution using the trained CTM. This process yields a vector for each sentence, where each element represents the probability of the sentence belonging to a specific topic.
2. *Similarity Measure* Calculate the similarity between the topic distribution vectors of the two sentences. Common methods to compute this similarity include:

- *Cosine Similarity* This measures the cosine of the angle between the two vectors. A higher cosine value indicates greater similarity.

The cosine similarity value obtained quantifies the degree of similarity between the two sentences based on their topic distributions. This value is then used for the sentence similarity analysis.

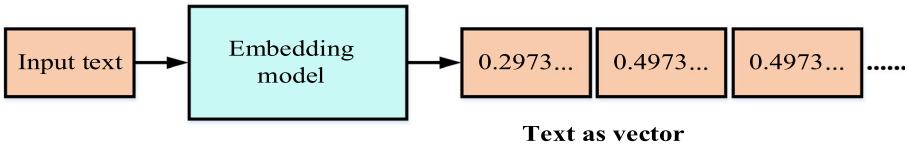

Fig. 2 Architecture of OpenAI embedding in text-to-word conversion

3.2.3 OpenAI

The OpenAI text embedding is used to evaluate the relatedness of the text strings. Open AI embedding is commonly utilized in clustering, recommendations, classification, searching, anomaly detection and diversity assessment tasks. OpenAI offers robust third-generation implanting models, such as "text-embedding-3-small", "text-embedding-3-large", and "text-embedding-ada-002". Text-embedding-ada-002 performs similarly to all prior embedding models in text, code search, and sentence comparison tests. Meanwhile, for classification, the Text-embedding-ada-002 outperformed previous models. This context length is raised fourfold, from 2048 to 8192 tokens, making it easier to handle large texts. These new embeddings, with 1536 dimensions and an extreme input length of 8191, are significantly smaller than Davinci embeddings. Semantically related words are allocated to vectors that are close together in a constant, dense, and low-dimensional vector space. This serves as the foundation for OpenAI embeddings [28, 29], which represent text using a Transformer NN. Figure 2 depicts the architecture of the OpenAI embedding for text-to-word conversion.

OpenAI embeddings are utilized in this study due to their ability to encode rich semantic and contextual information, making them more effective than traditional vectorization techniques like TF-IDF and Topic Modeling. By leveraging pre-trained deep learning models, these embeddings capture the underlying meaning of sentences rather than just their word frequency or topic distribution. This enables the model to correctly classify paraphrased sentences as similar, even when they share no common words. The impact of OpenAI embeddings is assessed through a comparative analysis with TF-IDF and Correlated Topic Models, ensuring a robust evaluation of similarity detection performance.

3.3 Model training

3.3.1 Cosine similarity (CS)

CS [30, 31] is a statistic used in natural language processing to determine how similar two sentences are. The cosine of the angle among two n-dimensional routes anticipated in multidimensional space is determined precisely using the CS measure. A text's CS runs from 0 to 1, with 1 indicating that two vectors have the same orientation and 0 indicating that two documents are less similar. The CS among two nonzero vectors is obtained using the following equation:

$$\text{Similarity}(S_1, S_2) = \frac{S_1 \cdot S_2}{S_1 * S_2} \tag{4}$$

where $S_1 \cdot S_2$ is the dot product of two sentence vectors, S_1 is the Euclidean norm of sentence vector S_1, and S_2 is the Euclidean norm of sentence S_2.

3.3.2 Bidirectional encoder representations from transformers-long short-term memory (BERT-LSTM)

In this, the BERT-LSTM model is used for the feature extraction process. BERT [32] paradigm is used for long-range processes. The BERT model is mainly used for masked language processes and to capture words from the next sentence, followed by a pre-trained model to increase the workflow of semantic tasks. The BERT technique only processes words of a maximum length of 768 tokens and can't be directly modified. LSTM [33] network model is used to filter facts from input values, and each input value has various degrees of effect in the final classification. This network model has input gates, output gates, and forgetting gates to hand-pick information and enhance memory units for loading and apprising information. The LSTM model works only on the order of relevant information, and that can't implement a parallel process. In this, the BERT-LSTM model is used to remove appropriate material from pre-processed data features and to solve feature dimensionality problems. The input layer comprises three phases: input of position vectors, word vectors, and segment vectors, which are then concatenated. First phase Word Vector: The model does not make word separation for English, Hindi and Hinglish text and directly uses one text as the base unit of text. The main feature information is contained in a word vector. The second phase is the position vector, which is related to the RNN; LSTM and BERT cannot determine the level of input word by model construction alone. So, this model enhances the location vector to remind word position. The third phase segment vector is used to discriminate among two input texts to meet the requirements of various activities. Figure 3 displays the structural design of the BERT-LSTM model.

BERT model output refers to the feature vectors at the sentence and character levels. Figure 3 depicts the model's leftmost output, the phrase vector, which encodes the semantics of the entire line. The leftover outputs are known as residual vectors, and each corresponds to a specific character. The BERT model includes a twelve-layer Transformer network, which is mostly used for the Transformer model's Encoder component. In the encoder section, attention workflow is usually utilized to analyze the input–output relationship and to investigate features that the surface network cannot culture through a deeper network layer. The BERT-LSTM model captures seep semantic and syntactic features. The BERT-LSTM model produces contextual embedding and consecutive patterns to feature-rich illustrations for long-range processes.

3.3.3 Convolutional neural network-long short-term memory (CNN-LSTM)

CNN and LSTM networks are two distinct deep learning architectures, each excelling in different aspects of data processing. While CNNs are highly effective at capturing spatial features, LSTMs are well-suited for modeling sequential dependencies. Their complementary strengths make them an ideal combination for text-processing tasks, such as sentence similarity detection, text classification, and machine translation. CNNs are primarily designed for processing spatial data, such as images, but they have also proven effective in text processing. When applied to text, CNNs use convolutional filters to capture local features, such as n-gram patterns, word dependencies, and positional relationships. This allows CNNs to extract high-level feature representations that are beneficial for downstream tasks. LSTM networks are a specialized type of recurrent neural network (RNN) designed to handle sequential dependencies. Unlike traditional RNNs, LSTMs mitigate the vanishing gradient problem by using memory cells and gating mechanisms, making them effective for long-range dependencies in

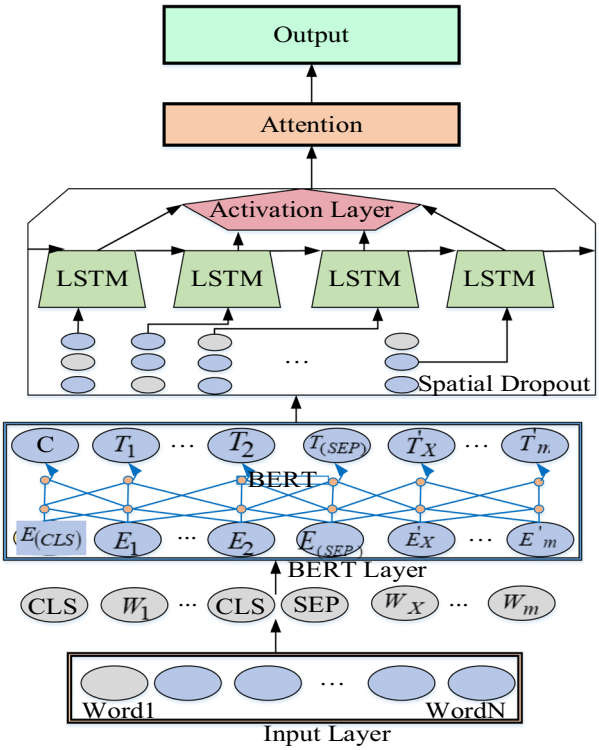

Fig. 3 Architecture of BERT-LSTM model

text data. CNN-LSTM models often outperform standalone CNN or LSTM models in NLP tasks due to their combined ability to extract both spatial and temporal information. This hybrid approach improves sentence similarity detection by ensuring that both local text patterns (CNN) and global sequential dependencies (LSTM) contribute to the decision-making process. CNNs [34, 35] provide benefits in a variety of NLP applications, including language modeling, autonomous conversion, and text classification. The CNN module, dubbed 1D-CNN, specializes in studying one-dimensional data structures and contains text. By stealing the filter from the input matrix, the convolutional layers intricated the input, eliminated structures, and passed the output layer to the following layer. CNN controls the size of an output matrix using two parameters: stride and padding. Stride signifies the number of pixels stimulated during the convolution process, whereas padding refers to the number of pixels added to an input matrix. These variables determine how the filter is implemented across the input matrix. The pooling layer in CNN aims to gradually decrease the spatial dimension of the feature maps generated by the previous convolution layer. This reduction is obtained by executing a pooling operation on the feature map. The dropout layer attempts to reduce overfitting by randomly setting input units to zero throughout each preparation period. After compressing an array into a one-dimensional array, the feed-forward NN takes it as input for further calculation.

LSTM [33] networks are a particular type of RNN intended to capture and use long-range needs within sequential data effectively. Standard RNNs produce drawbacks in capturing and using long-term dependencies within sequences because of vanishing gradient issues. As a

result, RNNs may struggle to accurately predict words that depend on context from distant past inputs. LSTM is a type of NN which obtains input (i_t) and outputs an assessment (o_t). The memory contains three gates, input, forget, and output, which control data flow and has the ability to add or remove data from the cell state, as specified by the horizontal line at the top of the diagram. The sigmoid layer creates a value among 0 and 1 after analysis, h_{t-1} and i_t to define which data is kept and is removed from the cell state.

$$f_t = \sigma\left(W_f\left[h_{t-1}, I_t\right] + b_f\right) \tag{5}$$

where f_t signifies forget gate at time t, σ is the sigmoid activation role, W_f stands for weight matrix, h_{t-1} is the hidden state at the earlier time step t-1, b_f bias term for forgetting gate and I_t input at the current time step t. The input gate is used to inform which part of the new candidate's assessment is written in the cell state. The sigma initiation role in the first unit expresses the desired part of the data. The Tanh initiation role used to map data among -1 and 1, obtains two standards in the second element. The input gate updates the cell state is defined by multiplying outputs of Tanh and the Sigma role.

$$x_t = \sigma\left(W_x\left[h_{t-1}, I_t\right] + b_x\right) \tag{6}$$

here x_t denotes input gate at time step t, σ sigmoid activation role, W_x weight matrix related to the input gate and b_x bias vector related to the input gate.

$$C_t = \tanh\left(W_c\left[h_{t-1}, I_t\right] + b_c\right) \tag{7}$$

where tanh stands for hyperbolic tangent activation role, W_c denotes the weight matrix and b_c bias vector associated with cell state.

$$C_t = F_t * C_{t-1} + i_t * C_t \tag{8}$$

Figure 4 shows the architecture of CNN-LSTM. In the last level, the output gate's sigmoid layer determines which components of the cell state are resumed as output.

The Tanh layer updates the cell state to an assessment among 1 and -1, and the ending output is created by multiplying the sigmoid output by the Tanh layer.

$$O_t = \sigma\left(W_o\left[h_{t-1}, I_t\right] + b_o\right) \tag{9}$$

$$h_t = O_t * \tanh(C_t) \tag{10}$$

3.3.4 Extreme gradient boosting (XGBoost)

XGboost [36] is a prominent boosting framework that combines regularization techniques to reduce overfitting issues. This is attained by $L1/L2$ penalties to weights and biases of each individual tree. The comparison score of root and leaf nodes is defined by using the following equation,

$$\text{Similarity Score} = \frac{\left(\sum \text{Residual}\right)^2}{N + \lambda} \tag{11}$$

where N denotes the number of residuals and λ stands for the regularization constraint. Gain is computed by using the following,

$$\text{Gain} = \text{Left Similarity} + \text{Right Similarity} - \text{Root Similarity} \tag{12}$$

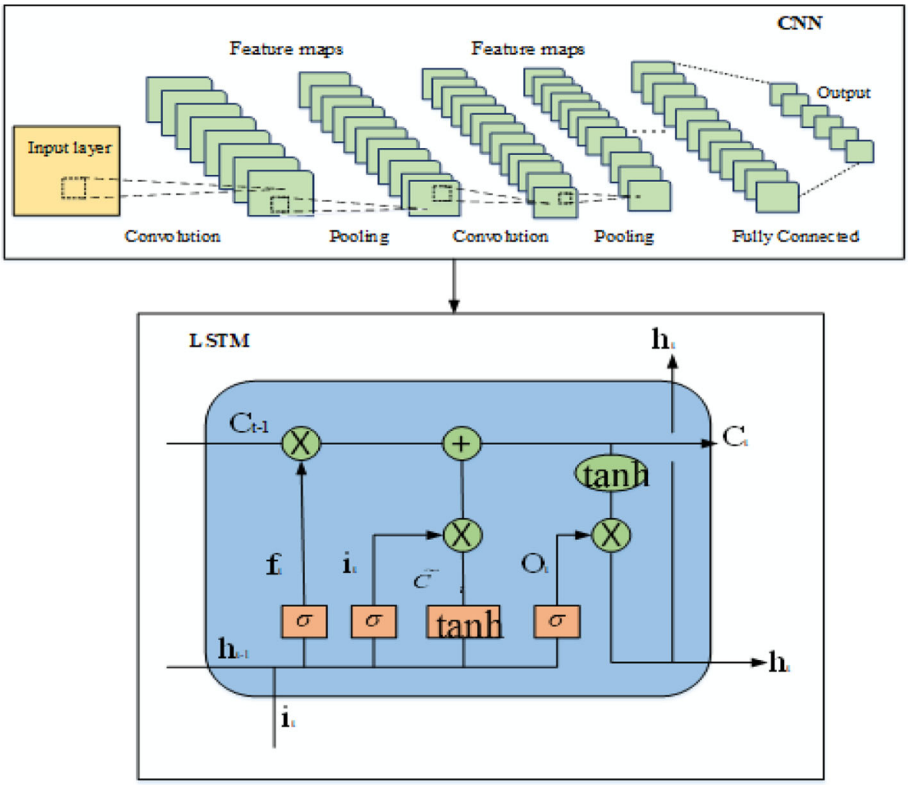

Fig. 4 CNN-LSTM architecture

The gamma parameter is used in the XGboost regression. If the expansion is a smaller amount of the gamma, the subdivision is severed, with no additional unbearable occurring or proceeding. Trimming is more common when the gamma value is greater. The XGboost learning part determines the model's convergence.

3.3.5 Adaptive boosting (AdaBoost)

Adaboost [37] is a sort of ensemble ML strategy that uses DT to build stumps. Adaboost combines weak beginners into a powerful classifier to increase the presentation of ML systems. Initially, the Adaboost algorithm builds the model by giving equal weights to all data points. The next version will prioritize learning from the previous model's misclassifications, giving these data points more weight. It will continue to train models until it has fewer faults. The final forecast is stated in the following equation:

$$\text{Final prediction} = \alpha_1 * p_1 + \alpha_2 * p_2 + \cdots + \alpha_n * p_n \tag{13}$$

where p_1 defines the model's calculation and α_1 define the model's amount of consequence.

4 Simulation results and analysis

This section shows the outcomes of the suggested approach with a detailed comparison.

4.1 Dataset description

To assess textual similarity, the suggested research analyzes 0.4 million Quora questions [38, 39]. Table 1 shows the properties of the Quora dataset. The dataset had 6000 samples for testing and 24,000 for training.

To provide a clearer view of the training data, Tables 2 and 3 present 5 sample sentence

Table 1 Attribute details of the Quora dataset

Attribute	Details
ID	Each dataset row has a unique identifier
Question_ID1, Question_ID2	'Question No. 1' and 'Question No. 2' contain questions with exclusive and differentiable identities
QuestionNo1, QuestionNo2	This feature combines actual questions to assess their degree of similarity
Is_Duplicate	When question sets are rationally surveyed, the result is Is_Replica; here, 0 and 1 indicate false and yes, respectively

Table 2 Sample sentence pairs from training dataset (class 0: not duplicate)

Index	Question 1	Question 2
1	How do I improve my English speaking?	What are some proven ways to become a better English speaker?
2	What's a good way to study for the exam?	What is the best strategy for exam preparation?
3	What are the benefits of exercise?	How does regular physical activity improve health?
4	Can you explain the theory of relativity?	What is Einstein's theory of general relativity?
5	What is the role of artificial intelligence?	How will AI impact the future?

Table 3 Sample sentence pairs from training dataset (class 1: duplicate)

Index	Question 1	Question 2
1	What is the purpose of a VPN?	Why should I use a VPN?
2	How do I bake a cake?	What's the recipe for baking a cake?
3	What are the symptoms of the flu?	How can you tell if you have influenza?
4	What is climate change?	What are the effects of global warming?
5	How does photosynthesis work?	Can you explain the process of photosynthesis?

pairs from class 0 (not duplicate) and 5 samples from class 1 (duplicate), respectively.

To further illustrate the feature extraction process, it provides the vector representations of the sentences in Tables 2 and 3 using TF-IDF, OpenAI embeddings, and Topic Modeling. For brevity, due to the high dimensionality of these vectors, they will show a truncated representation, displaying only the first 5 dimensions. However, it is important to note that the actual vectors used in the model are full-dimensional. Truncated Vector Representations of Sample Sentences are given in Table 4.

4.2 Performance metrics

Table 5 describes mathematical expressions for various performance metrics like accuracy, precision, and so on.

TN describes true negative rate, TP means true positive rate, FP denotes false positive rate and FN stands for false negative rate. Also, n define the number of classes, AP_k average precision of class k.

4.3 Evaluation results

This section shows the graphical representation of various classification metrics using multiple techniques. Figure 5 depicts the classification assessment metrics for the various approaches that were learned using TF-IDF.

Here, compared to other models, XGBoost produced superior results in terms of 95.34% accuracy, 94.85% precision, 95.268% recall, 95.5% F-measure, and 95.52% mAP. The Adaboost model performs poorly when compared to other models, with 90.4% accuracy, 90.5% precision, 90.568% recall, 89% F-measure, and 89.1% mAP. Figure 6 displays the performance of all models trained with OpenAI embeddings.

The graph shows that the BERT-LSTM technique outperforms other models, such as cosine similarity, XGboost, Adaboost, and CNN-LSTM. BERT-LSTM yields findings of 95.4% accuracy, 95.68% recall, 95.5% precision, 95.6% F-measure, and 95.87% mAP. Meanwhile, the AdaBoost approach shows low-performance results in terms of 90.4% accuracy, 90.5% precision, 90.68% recall, 90% F-measure, and 90.1% mAP. Performance analysis based on the correlated topic model is given in Fig. 7.

Compared to other techniques, the Cosine Similarity approach shows good results in terms of 94.1% accuracy, 94.2% precision, 94.968% recall, 94.5% F-measure and 94.61% mAP. The AdaBoost is producing unsatisfactory results with an accuracy of 90.4%, recall of 90.68%, precision of 90.5%, F-measure of 90% and mAP of 90.1%. FPR performance comparison for models using embedding techniques is given in Fig. 8.

Here, the graphs describe FPR performance comparison using embedding techniques, namely TF-IDF, OpenAI, and correlated topic model. In TF-IDF, compared to other models, XGBoost shows a lower FPR value, 0.22. BERT-LSTM shows less FPR value of 0.25 in OpenAI, which is low compared to other models. For the CTM, the Cosine Similarity model produces less FPR value of 0.26. Performance comparison based on embedding techniques is specified in Table 6. Table 7 shows performance analysis without pre-processing.

Table 8 shows the execution time of each model across embedding techniques. DL models (BERT-LSTM and CNN-LSTM) were executed using GPU acceleration. Classical models were trained on the CPU. Cosine Similarity is not a trainable model; hence, its execution time is negligible and depends mainly on matrix similarity calculations. OpenAI embeddings were pre-generated and loaded during training to reduce vectorization time. Table 9 shows

Table 4 Truncated vector representations of sample sentences

Index	Class	Feature	Question 1 Vector (Truncated)	Question 2 Vector (Truncated)
1	0	TF-IDF	[0.12, 0.0, 0.23, 0.0, 0.05]	[0.08, 0.0, 0.15, 0.21, 0.0]
1	0	OpenAI	[0.01, 0.02, − 0.01, 0.03, − 0.02]	[0.02, 0.01, 0.0, 0.02, − 0.01]
1	0	Topic Modeling	[0.2, 0.1, 0.05, 0.3, 0.15]	[0.15, 0.2, 0.1, 0.25, 0.05]
2	0	TF-IDF	[0.0, 0.15, 0.0, 0.20, 0.0]	[0.0, 0.10, 0.18, 0.0, 0.05]
2	0	OpenAI	[− 0.01, 0.03, 0.02, − 0.02, 0.01]	[0.0, 0.02, − 0.01, 0.01, − 0.03]
2	0	Topic Modeling	[0.25, 0.05, 0.1, 0.2, 0.4]	[0.3, 0.1, 0.0, 0.3, 0.3]
3	0	TF-IDF	[0.22, 0.0, 0.0, 0.18, 0.0]	[0.18, 0.0, 0.25, 0.0, 0.07]
3	0	OpenAI	[0.02, − 0.01, 0.01, 0.0, − 0.02]	[0.03, 0.0, − 0.02, 0.01, 0.01]
3	0	Topic Modeling	[0.1, 0.2, 0.3, 0.1, 0.3]	[0.2, 0.1, 0.2, 0.2, 0.3]
4	0	TF-IDF	[0.0, 0.0, 0.25, 0.0, 0.15]	[0.0, 0.21, 0.0, 0.0, 0.12]
4	0	OpenAI	[0.01, 0.02, 0.0, − 0.03, 0.02]	[− 0.02, 0.01, 0.03, 0.0, − 0.01]
4	0	Topic Modeling	[0.3, 0.1, 0.2, 0.2, 0.2]	[0.25, 0.15, 0.3, 0.1, 0.2]
5	0	TF-IDF	[0.18, 0.0, 0.0, 0.22, 0.0]	[0.15, 0.0, 0.0, 0.0, 0.28]
5	0	OpenAI	[− 0.02, 0.01, − 0.01, 0.02, 0.0]	[0.0, − 0.02, 0.02, − 0.01, 0.03]
5	0	Topic Modeling	[0.2, 0.3, 0.1, 0.2, 0.2]	[0.15, 0.25, 0.2, 0.1, 0.3]
1	1	TF-IDF	[0.25, 0.0, 0.18, 0.0, 0.0]	[0.20, 0.0, 0.15, 0.22, 0.0]
1	1	OpenAI	[0.03, − 0.01, 0.02, 0.01, − 0.02]	[0.02, 0.0, 0.01, − 0.02, 0.01]
1	1	Topic Modeling	[0.1, 0.3, 0.2, 0.2, 0.2]	[0.2, 0.25, 0.15, 0.2, 0.2]
2	1	TF-IDF	[0.0, 0.22, 0.0, 0.18, 0.0]	[0.20, 0.0, 0.25, 0.0, 0.05]
2	1	OpenAI	[− 0.01, 0.02, 0.0, 0.03, − 0.01]	[0.0, 0.01, − 0.02, 0.02, − 0.03]
2	1	Topic Modeling	[0.3, 0.2, 0.05, 0.25, 0.2]	[0.25, 0.3, 0.1, 0.2, 0.15]
3	1	TF-IDF	[0.18, 0.0, 0.25, 0.0, 0.07]	[0.22, 0.0, 0.0, 0.18, 0.0]

Table 4 (continued)

Index	Class	Feature	Question 1 Vector (Truncated)	Question 2 Vector (Truncated)
3	1	OpenAI	[0.02, 0.01, − 0.02, 0.0, 0.01]	[0.01, 0.0, 0.0, 0.01, − 0.02]
3	1	Topic Modeling	[0.2, 0.1, 0.2, 0.3, 0.2]	[0.1, 0.2, 0.3, 0.1, 0.3]
4	1	TF-IDF	[0.0, 0.0, 0.21, 0.0, 0.12]	[0.0, 0.0, 0.25, 0.0, 0.15]
4	1	OpenAI	[0.01, − 0.02, 0.03, 0.0, − 0.01]	[− 0.02, 0.01, 0.02, − 0.01, 0.0]
4	1	Topic Modeling	[0.25, 0.15, 0.3, 0.1, 0.2]	[0.3, 0.1, 0.2, 0.2, 0.2]
5	1	TF-IDF	[0.15, 0.0, 0.0, 0.0, 0.28]	[0.18, 0.0, 0.0, 0.22, 0.0]
5	1	OpenAI	[0.0, 0.02, − 0.01, 0.03, − 0.03]	[− 0.02, 0.03, − 0.02, 0.01, 0.0]
5	1	Topic Modeling	[0.1, 0.2, 0.2, 0.1, 0.4]	[0.2, 0.1, 0.1, 0.3, 0.3]

Table 5 Mathematical expression for various performance metrics

Metrics	Formula
Accuracy	$\frac{\mathrm{TP+TN}}{\mathrm{TP+TN+FP+FN}}$
Precision	$\frac{\mathrm{TP}}{\mathrm{TP+FP}}$
Recall	$\frac{\mathrm{TP}}{\mathrm{TP+FN}}$
mAP	$\frac{1}{n}\sum_{k=1}^{k=n}\mathrm{AP}_k$
F-measure	$2 * \frac{[\mathrm{Precision}*\mathrm{Recall}]}{[\mathrm{Precision}*\mathrm{Recall}]}$
FPR	$\mathrm{FP}/(\mathrm{FP}+\mathrm{TN})$
MCC	$\frac{\mathrm{TP}\times\mathrm{TN}-\mathrm{FP}-\mathrm{FN}}{(\mathrm{TP+FP})(\mathrm{TP+FN})(\mathrm{TN+FP})(\mathrm{TN+FN})}$
Cohen's Kappa	$\frac{p_o-p_c}{1-p_c}$
MAE	$\frac{1}{n}\sum_{i=1}^{n}\lvert y_i-\hat{y}_i\rvert$
RMSE	$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}$

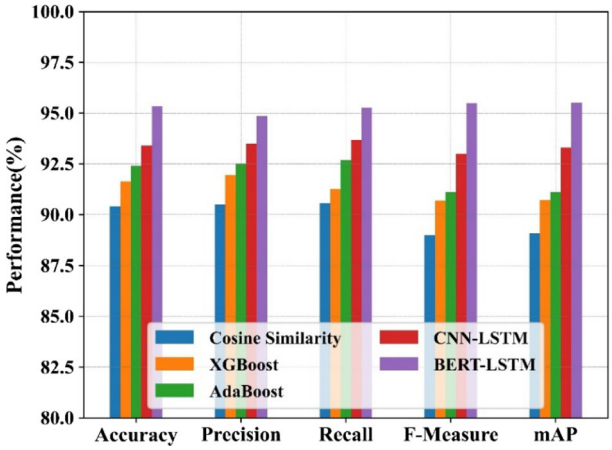

Fig. 5 Classification assessment metrics for the various methods that were trained on TF-IDF

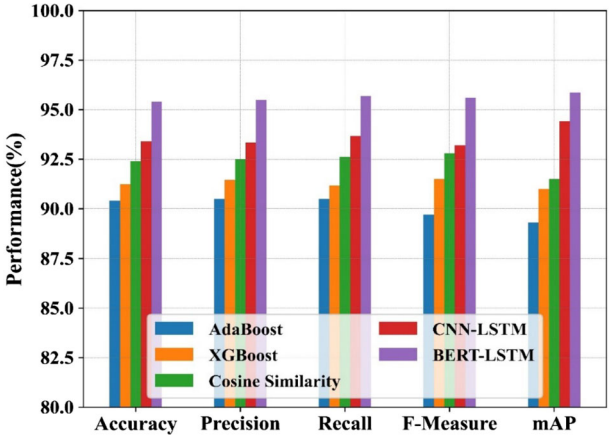

Fig. 6 The performance of OpenAI embedding

the benchmark comparison analysis.

5 Conclusion

Sentence similarity detection has broad applications, consisting of question-answering systems, community forms, e-commerce customer help, education fields and customer chatboxes. Automating this process is necessary for promptly fulfilling user expectations and improving their overall experience. OpenAI embedding provides advantages in various fields, namely search, clustering, comparison analysis, anomaly recognition, diversity assessment, and classification. The proposed findings define that ML models leverage the information encoded within these embeddings effectively, which leads to significant improvement in prediction accuracy. The proposed study highlights the strong usage of OpenAI embeddings

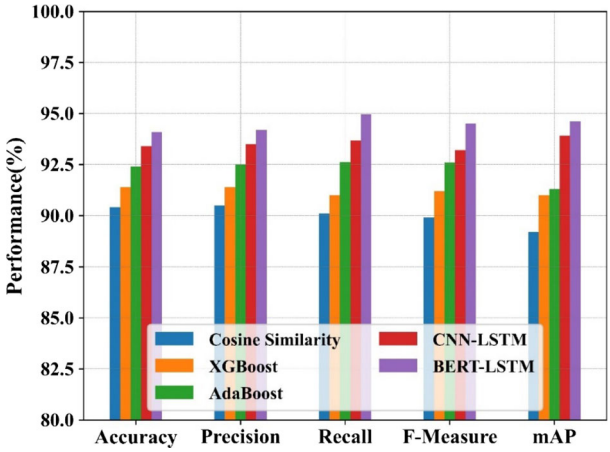

Fig. 7 Performance of Correlated Topic Model

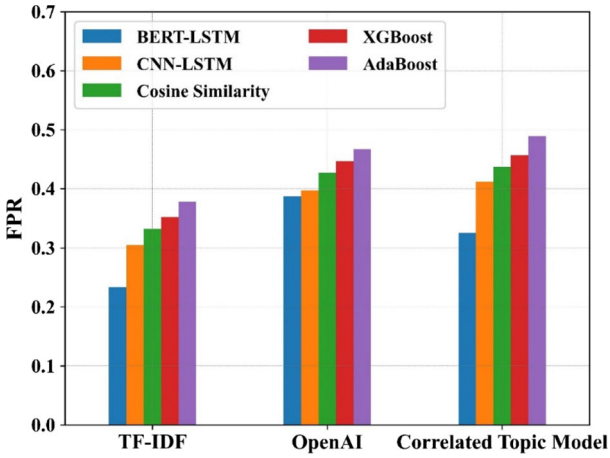

Fig. 8 FPR performance comparison for models using embedding techniques

and CNN models by high-performance results. Future research will refine these models by optimizing CNN parameters and incorporating speech-to-text and text-to-speech (TTS) capabilities to support users in interrelating with the system through voice input and receiving verbal responses.

Table 6 Comparison based on embedding techniques using with pre-processing

Methods	Accuracy	Precision	Recall	F-measure	mAP	FPR	MCC	MAE	Cohen's Kappa	RMSE
TF-IDF										
BERT-LSTM	95.34	94.85	95.268	95.5	95.52	0.22	0.92	0.045	0.91	0.032
CNN-LSTM	93.4	93.5	93.68	93	93.3	0.322	0.88	0.067	0.86	0.058
AdaBoost	92.4	92.5	92.68	91.1	91.1	0.387	0.85	0.078	0.83	0.072
XGBoost	91.64	91.95	91.268	90.7	90.71	0.42	0.82	0.089	0.81	0.085
Cosine Similarity	90.4	90.5	90.568	89	89.1	0.487	0.79	0.102	0.78	0.098
OpenAI										
BERT-LSTM	95.4	95.5	95.68	95.6	95.87	0.25	0.93	0.042	0.92	0.03
CNN-LSTM	93.4	93.35	93.68	93.2	94.41	0.35	0.89	0.065	0.87	0.056
Cosine Similarity	92.4	92.5	92.628	92.8	91.5	0.415	0.86	0.077	0.84	0.071
XGBoost	91.24	91.45	91.168	91.5	91	0.389	0.83	0.088	0.82	0.083
AdaBoost	90.4	90.5	90.5	89.7	89.3	0.478	0.8	0.099	0.79	0.096
CTM										
BERT-LSTM	94.1	94.2	94.968	94.5	94.61	0.26	0.91	0.044	0.9	0.031
CNN-LSTM	93.4	93.5	93.68	93.2	93.92	0.395	0.88	0.067	0.86	0.057
AdaBoost	92.4	92.5	92.628	92.6	91.3	0.422	0.85	0.079	0.83	0.073
XGBoost	91.4	91.4	91	91.2	91	0.32	0.82	0.09	0.81	0.086
Cosine Similarity	90.4	90.5	90.1	89.9	89.2	0.492	0.79	0.103	0.78	0.099

Table 7 Performance analysis of proposed and previous models without pre-processing

Methods	Accuracy	Precision	Recall	F-measure	mAP	FPR	MCC	MAE	Cohen's Kappa	RMSE
TF-IDF										
BERT-LSTM	95.1	94.9	95.0	95.1	95.20	0.25	0.91	0.045	0.89	0.035
CNN-LSTM	93.2	93.1	93.3	93.2	93.10	0.35	0.85	0.070	0.84	0.060
AdaBoost	92.5	92.4	92.6	92.5	91.00	91.00	0.83	0.080	0.82	0.074
XGBoost	91.8	91.6	91.7	91.8	0.80	0.090	0.80	0.090	0.79	0.085
Cosine Similarity	90.2	90.1	90.3	90.2	89.80	0.50	0.78	0.095	0.76	0.099
OpenAI										
BERT-LSTM	95.3	95.0	95.2	95.3	95.50	0.22	0.93	0.040	0.91	0.032
CNN-LSTM	93.4	93.2	93.3	93.4	94.00	0.30	0.88	0.055	0.87	0.050
AdaBoost	92.6	92.5	91.8	92.6	92.50	0.38	0.85	0.075	0.83	0.068
XGBoost	91.9	91.7	90.5	91.9	91.00	0.42	0.83	0.080	0.81	0.070
Cosine Similarity	90.4	90.3	95.3	90.4	90.00	0.48	0.79	0.085	0.78	0.095
CTM										
BERT-LSTM	94.9	94.7	94.8	94.9	94.90	0.26	0.91	0.041	0.90	0.031
CNN-LSTM	93.0	92.9	93.0	93.0	93.20	0.35	0.87	0.057	0.86	0.055
AdaBoost	92.3	92.2	92.4	92.3	91.30	0.40	0.85	0.073	0.83	0.070
XGBoost	91.5	91.4	91.6	91.5	91.20	0.42	0.82	0.078	0.81	0.086
Cosine Similarity	90.0	89.9	90.1	90.0	89.90	0.49	0.79	0.098	0.78	0.099

Table 8 Execution time of each model across embedding techniques

Model	TF-IDF (min)	OpenAI (min)	CTM (min)
BERT-LSTM	30	28	29
CNN-LSTM	18	17	18
AdaBoost	1.2	1.4	1.3
XGBoost	0.9	1.0	0.95
Cosine Similarity	~ 0.02	~ 0.02	~ 0.02

Table 9 Performance comparison of best model with relevant benchmark

Models	F-Measure
Siamese LSTM with Blend of word embeddings [22]	82.41
QAMPARI [40]	32.8
Factuality or Fiction [41]	68.2
Proposed (BERT-LSTM)	95%

Authors contributions *1. Mrs. Gayatri Girish Asalkar: Writing—Original Draft Preparation, Specifically Writing the Initial Draft.Writing—Review & Editing Preparation, Specifically Critical Review, Commentary. Writing—Original Draft Preparation, Specifically Writing the Initial Draft.Writing—Review. 2. Dr. Bechoo Lal: Specifically Critical Review, Commentary. Writing—Original Draft Preparation, Specifically Writing the Initial Draft.Writing—Review. 3. Dr. Nilesh B. Korade: Writing—Original Draft Preparation, Specifically Writing the Initial Draft. Writing—Review & Editing Preparation. All authors reviewed the manuscript.

Funding No funding is provided for the preparation of the manuscript.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no competing interests.

Ethical approval This article does not contain any studies with human participants or animals performed by any of the authors.

Consent to participate All the authors involved have agreed to participate in this submitted article.

Consent to Publish All the authors involved in this manuscript give full consent for publication of this submitted article.

References

1. de Oliveira RS, Nascimento EG (2022) Analyzing similarities between legal court documents using natural language processing approaches based on Transformers. arXiv preprint https://arxiv.org/abs/2204.07182
2. Shabariram CP, Priya Ponnuswamy P (2023) Semantic similarity based automated answer script evaluation system using machine learning pipeline and natural language processing. In: Computational vision and bio-inspired computing: proceedings of ICCVBIC 2022. Springer, Singapore, pp 495–509

3. Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, Agarwal S (2020) Language models are few-shot learners. Adv Neural Inf Process Syst 33:1877–1901
4. Radford A. Improving language understanding by generative pre-training
5. Su J, Ahmed M, Lu Y, Pan S, Bo W, Liu Y (2024) Roformer: Enhanced transformer with rotary position embedding. Neurocomputing 568:127063
6. Reimers N (2019) Sentence-BERT: sentence embeddings using siamese BERT-networks. arXiv preprint https://arxiv.org/abs/1908.10084
7. Conneau A (2019) Unsupervised cross-lingual representation learning at scale. arXiv preprint https://arxiv.org/abs/1911.02116
8. Liu W, Zhou P, Zhao Z, Wang Z, Ju Q, Deng H, Wang P (2020) K-bert: Enabling language representation with knowledge graph. Proc AAAI Conf Artif Intell 34(03):2901–2908
9. Xu T, Zhang F (2020) A brief review of relation extraction based on pre-trained language models. Fuzzy Syst Data Min 6:775–789
10. Pérez A, Parapar J, Barreiro Á (2022) Automatic depression score estimation with word embedding models. Artif Intell Med 132:102380
11. Yu R, Lu W, Lu H, Wang S, Li F, Zhang X, Yu J (2021) Sentence pair modeling based on semantic feature map for human interaction with IoT devices. Int J Mach Learn Cybern 12(11):3081–3099
12. Amur ZH, Kwang Hooi Y, Bhanbhro H, Dahri K, Soomro GM (2023) Short-text semantic similarity (stss): techniques, challenges and future perspectives. Appl Sci 13(6):3911
13. Sun K, Luo X, Luo MY (2022) A survey of pretrained language models. In: International conference on knowledge science, engineering and management. Springer, Cham, pp 442–456
14. Alsuhaibani M (2023) Deep learning-based sentence embeddings using BERT for textual entailment. Int J Adv Comput Sci Appl 14(8)
15. Feng J, Liu J, Gu C, Qi H, Ren Z, Xu K, Wang Y (2024) Focusing on differences! Sample framework enhances semantic textual similarity with external knowledge. Expert Syst Appl 124462
16. Ahmad S, Saqib SM, Syed AH (2024) CNN and LSTM based hybrid deep learning model for sentiment analysis on arabic text reviews. Mehran Univ Res J Eng Technol 43(2):183–194
17. Alyamani HJ, Ahmad S, Syed AH, Saqib SM, Al-Otaibi YD (2021) LSI authentication-based Arabic to English text converter. Int J Fuzzy Logic Intell Syst 21(4):409–422
18. Saqib SM, Ahmad S, Syed AH, Naeem T, Alotaibi FM (2019) Analysis of latent Dirichlet allocation and non-negative matrix factorization using latent semantic indexing. Int J Adv Appl Sci 6(10):94–102
19. Saqib SM, Naeem T, Ahmad S, Alorfi AS (2023) Evaluation of deep learning approaches for sentiment analysis. VAWKUM Trans Comput Sci 11(1):26–41
20. Korade NB, Salunke MB, Asalkar GG, Khedkar RG, Bhosale AU, Joshi DM, Jadhav AC. Intelligent systems and applications in engineering
21. Le HT, Cao DT, Bui TH, Luong LT, Nguyen HQ (2021) Improve quora question pair dataset for question similarity task. In: 2021 RIVF international conference on computing and communication technologies (RIVF). IEEE, pp 1–5
22. Imtiaz Z, Umer M, Ahmad M, Ullah S, Choi GS, Mehmood A (2020) Duplicate questions pair detection using siamese malstm. IEEE Access 8:21932–21942
23. Nijagunarya Y. Detecting duplicate questions in community based websites using machine learning
24. Wang L, Zhang L, Jiang J (2020) Duplicate question detection with deep learning in stack overflow. IEEE Access 8:25964–25975
25. Wang L, Liu S, Qiao L, Sun W, Sun Q, Cheng H (2022) A cross-lingual sentence similarity calculation method with multifeature fusion. IEEE Access 10:30666–30675
26. Wu MJ, Fu TY, Chang YC, Lee CW (2020) A study on natural language processing classified news. In: 2020 Indo–Taiwan 2nd international conference on computing, analytics and networks (Indo-Taiwan ICAN). IEEE, pp 244–247
27. Lan F (2022) Research on text similarity measurement hybrid algorithm with term semantic information and TF-IDF method. Adv Multimedia 2022(1):7923262
28. Ciaccio EJ (2023) Use of artificial intelligence in scientific paper writing. Inform Med Unlocked 41:101253
29. Korade NB, Salunke MB, Bhosle AA, Asalkar GG, Lal B, Kumbharkar PB (2025) Elevating intelligent voice assistant chatbots with natural language processing, and OpenAI technologies. Indonesian J Electr Eng Comput Sci 37(1):507–517
30. Ansorena IL (2021) On the benchmarking of port performance: a cosine similarity approach. Int J Process Manag Benchmark 11(1):101–114
31. Ramya RS, Singh G, Sejal SN, Venugopal KR, Iyengar SS, Patnaik LM (2021) R 2 DCLT: retrieving relevant documents using cosine similarity and LDA in text mining. Int J Inf Commun Technol 19(4):391–422

32. Alammary AS (2022) BERT models for Arabic text classification: a systematic review. Appl Sci 12(11):5720
33. Saha BN, Senapati A (2020) Long short term memory (lstm) based deep learning for sentiment analysis of english and spanish data. In: 2020 international conference on computational performance evaluation (ComPE). IEEE, pp 442–446
34. Korade NB, Zuber DM (2023) Boost stock forecasting accuracy using the modified firefly algorithm and multichannel convolutional neural network. J Theor Appl Inf Technol 15(7)
35. Korade NB, Zuber M (2022) Stock price forecasting using convolutional neural networks and optimization techniques. Int J Adv Comput Sci Appl 13(11)
36. Wang C, Xu S, Yang J (2021) Adaboost algorithm in artificial intelligence for optimizing the IRI prediction accuracy of asphalt concrete pavement. Sensors 21(17):5682
37. Alghazzawi DM, Alquraishee AG, Badri SK, Hasan SH. ERF-XGB: Ensemble Random Forest-Based XG Boost for Accurate Prediction and Classification of E-Commerce Product Review: Sustainability
38. Korade NB, Salunke MB, Bhosle AA, Kumbharkar PB, Asalkar GG, Khedkar RG (2024) Strengthening sentence similarity identification through openAI embeddings and deep learning. Int J Adv Comput Sci Appl 15(4)
39. Korade NB, Salunke MB, Asalkar GG, Khedkar RG, Bhosale AU, Joshi DM, Jadhav AC (2024) Exploring NLP techniques for duplicate question detection to maximizing responses on Q&A websites. Int J Intell Syst Appl Eng 12(3):11–20
40. Amouyal S, Wolfson T, Rubin O, Yoran O, Herzig J, Berant J (2023) QAMPARI: a benchmark for open-domain questions with many answers. In: Proceedings of the third workshop on natural language generation, evaluation, and metrics (GEM), pp 97–110
41. Patel M, Anand A (2024) Factuality or fiction? Benchmarking modern LLMs on ambiguous QA with citations. arXiv preprint https://arxiv.org/abs/2412.18051

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Gayatri G. Asalkar is pursuing a Ph.D. from Shri Jagdishprasad Jhabarmal Tibrewala University, Rajasthan, India, and is an Assistant Professor at JSPM's Rajarshi Shahu College of Engineering, Pune, India. Her research areas are data science, data analysis, time series, machine learning, NLP, and deep learning. She has worked on several consultancy projects, published several research papers, and obtained copyrights for her creative ideas.

Bechoo Lal is an Associate Professor at KL University, Andhra Pradesh, India. He holds a Ph.D. in Computer Science, a Ph.D. in Information Systems, and a PGP in Data Science from Purdue University, USA. He is a supervising Ph.D. research scholar from SJJT University, Rajasthan, and a member of the International Association of Engineers, USA. His research areas are data science, big data analytics, and machine learning.

Nilesh B. Korade is an Assistant Professor at Department of Computer Science and Engineering (Artificial Intelligence), Vishwakarma Institute of Technology, Pune, India. He is a data science professional with a Ph.D. in Computer Science and Engineering. His research areas are data science, data analysis, time series, machine learning, NLP, and deep learning. He has worked on several consultancy projects, filed several Indian patents, and obtained copyrights for his creative ideas.